DFA Implementation Using C++

1. Introduction

This program implements a Deterministic Finite Automaton (DFA) in C++ that is obtained from a Regular Expression (RE) through ε-NFA construction followed by subset construction (NFA → DFA).

The DFA processes strings consisting of symbols {a, b} and determines whether a given input string is ACCEPTED or REJECTED based on predefined final states.

2. Objectives

The objectives of this program are:

- To simulate a DFA using C++
- To check whether a given input string is accepted by the DFA
- To implement DFA transition logic using functions
- To verify acceptance using final state detection

3. Theory Overview

Regular Expression to NFA

- The given Regular Expression is first converted into an ε-NFA using Thompson's construction.
- ε-closures of NFA states are calculated.

NFA to DFA

- The ε-NFA is converted into DFA using subset construction.
- Each DFA state represents a set of NFA states.
- DFA final states are those that contain NFA final states (11 or 24)

4. How the Program Takes Input and Processes the String

First, the program asks the user to enter a string. Then it takes the input using cin and store it in a string variable named input. The DFA always starts from state 0, which is the initial state. Then the program reads the string character by character.

The moveState() function to control the movement between DFA states.

- This function takes the current state and the input character.
- Based on the DFA transition rules, it returns the next state.
- I used if conditions to represent each transition.
- This function is called for every character of the input string.
- It helps the program follow the DFA correctly.

5. State Transition Handling

To control the movement between DFA states, a separate transition mechanism is used.
For every character in the input string:

- The current state of the DFA is checked
- The input character is analysed
- Based on the DFA transition rules, the DFA moves to the next state

This process ensures that the DFA follows the correct path defined by the transition diagram.
Because the automaton is deterministic, each combination of current state and input symbol leads to only one next state.

6. Role of the Accepting State Checking

After the entire input string has been processed, the DFA reaches a final state.
At this point, the program checks whether the final state is an accepting state or not.
Certain DFA states are predefined as accepting states because they contain NFA final states (11 or 24). If the DFA ends in any one of these accepting states, the input string is considered valid.

7. Final Decision of the Program

Once the acceptance condition is checked:

- If the final state is an accepting state, the program displays "String ACCEPTED"
- If the final state is not an accepting state, the program displays "String REJECTED"

This decision is made only after processing the full input string.

8. Applications

The implemented DFA can be used in:

- Lexical analysis in compilers
- Pattern recognition systems
- String validation problems
- Formal language processing

8. Conclusion

This implementation successfully demonstrates how a Deterministic Finite Automaton can be used to validate strings. The DFA is constructed from a Regular Expression through ε-NFA and subset construction techniques. The program accurately checks string validity using DFA transitions and final state evaluation.
This approach is efficient, deterministic, and suitable for practical applications in automata theory and compiler design.